

John Holik

Chicago, IL · 239-989-2566 · jholik.dev@gmail.com · linkedin.com/in/john-holik-iii · github.com/John-Holik

Software engineer focused on applied machine learning: probabilistic modeling, uncertainty quantification, and rigorous model evaluation, plus LLM application development. Second author on a manuscript under review at the *Journal of Management in Engineering* (ASCE); sole developer of a four-method ensemble uncertainty-quantification pipeline published as a public Jupyter Book; shipped a live retrieval-augmented chatbot (ElectriAI, on Cloudflare). Strengths in evaluating models against ground truth, calibrated probabilistic forecasting, and reproducible experimentation.

EDUCATION

Florida Gulf Coast University, Fort Myers, FL

May 2026

B.S. in Computer Software Engineering · Dean's List (Junior and Senior Years)

Relevant coursework: Data Structures and Algorithms, Machine Learning, Data Mining, Data Engineering, Operating Systems, Computer Security, Software Engineering, AI Entrepreneurship

TECHNICAL SKILLS

Languages: Python, C++, SQL, Bash, PowerShell

ML and Data: scikit-learn, pandas, NumPy, SciPy, TensorFlow/Keras, ensemble methods and data assimilation (Ensemble Kalman Filter, GLUE, residual bootstrap), time-series forecasting (LSTM), uncertainty quantification and probabilistic evaluation (CRPS, coverage, spread-skill, calibration), classification metrics (ROC-AUC, precision/recall/F1, Cohen's kappa), clustering (K-means, UMAP, HDBSCAN)

AI and LLM: OpenAI, Anthropic (Claude), and Gemini APIs, prompt engineering, structured JSON-schema output, LLM evaluation vs human ground truth, custom agent and Claude Code skill development, RAG and embeddings (deployed retrieval-augmented chatbot)

Web and Backend: React, FastAPI, Node.js/Express, PostgreSQL, Cloudflare Pages/Workers (deployed)

Tools: Git/GitHub, Jupyter, pytest and Hypothesis (property-based testing), matplotlib/plotly, LaTeX, Linux

EXPERIENCE

Undergraduate Research Assistant, then Independent Researcher (ElectriAI)

Aug 2025 – Present

U.A. Whitaker College of Engineering, Florida Gulf Coast University

- Built an end-to-end LLM pipeline over a dataset of electrical-construction YouTube videos and comment threads: collection, schema-constrained GPT classification, human annotation, and rigorous evaluation. Second author on the resulting manuscript, "Identifying Knowledge Bottlenecks in Electrical Construction from Practitioner Discussion on YouTube Using Large Language Models," submitted to the *Journal of Management in Engineering* (ASCE) July 2026.
- Processed 794 video transcripts and 66,899 comment threads into 16,862 structured question-and-answer records under a 76-theme, ten-category taxonomy; classified 12,933 questions into ten functional forms.
- Validated model labels against an independent 26-annotator human consensus: 0.847 Cohen's kappa, 86.6% accuracy, 0.858 weighted F1, with per-class precision, recall, and F1.
- Built a reproducible evaluation harness (fixed seeds, versioned data) and released the supporting data and code as a citable Zenodo archive.
- Built and deployed the public ElectriAI companion web app (React, Tailwind) on Cloudflare Pages, and generated its chatbot knowledge base from the taxonomy rather than hand-authoring it: 288 pages (263 question-family, 11 question-type, 10 answer-type, 3 analytics, 1 overview), each with computed answer-rate and per-year statistics and verbatim threads carrying (V:videoID)/(Q:commentID) provenance. Chunked under a 6,000-character cap, splitting long pages at H2 boundaries with the page title prefixed to each chunk for context: 297 chunks, embedded offline via gemini-embedding-001 at 768 dimensions (RETRIEVAL_DOCUMENT) and shipped as static JSON, so no vector database is required.
- Built the retrieval-to-answer path: cosine similarity over the 297 vectors plus a keyword-overlap boost and regex intent routing to the analytics pages, capped at two chunks per page for diversity, top eight retained. The prompt renders the top four in full and the next four as 320-character snippets, each tagged with its score, and constrains the model to those passages, verbatim citation tokens, and no number absent from them; a post-generation regex check strips ungrounded numbers and retries. Streams from gemini-3.6-flash through a Cloudflare Worker proxy (API-key isolation, KV rate limiting). Live at youtube.electriai.com.

PROJECTS

Red Tide Reanalysis, Senior Capstone

Python, NumPy, SciPy, scikit-learn, TensorFlow, spotpy, pytest/Hypothesis · john-holik.github.io/redtide-reanalysis-book

- Sole developer of all code on a four-person capstone team (268 commits across five repositories). Quantified how uncertainty in modeled river inputs propagates into a *Karenia brevis* (red tide) bloom classifier for the Peace River, Florida.
- Implemented four structurally different uncertainty-quantification methods generating 200-member ensembles for river discharge and total nitrogen: residual bootstrap with a Durbin-Watson diagnostic auto-routing IID versus AR(1) resampling, GLUE over 10,000 Monte-Carlo parameter draws filtered by a Nash-Sutcliffe likelihood threshold, an **Ensemble Kalman Filter written from scratch in NumPy** with covariance inflation and heteroscedastic observation error, and analytic Jacobian-covariance propagation with a condition-number check and pseudo-inverse fallback.
- Scored every ensemble on probabilistic and deterministic metrics (CRPS via a sort-based $O(N \log N)$ formulation, CRPS skill score, coverage probability, spread-skill ratio, rank histograms, KGE, NSE), then propagated each member through a pre-trained Random Forest classifier to turn a deterministic bloom prediction into a distribution.
- **Key result:** substituting EnKF-reanalyzed discharge for real observations cost 0.004 balanced accuracy (0.884 versus a 0.887 observed-data baseline), showing the reanalysis can stand in for observations in periods where none exist. Identified that the dominant failure mode was recall, not precision (0.695 and 0.520 for the worst-calibrated ensembles against ~0.88 precision throughout).
- Diagnosed why two methods failed on total nitrogen (coverage 0.27 and 0.32 against a nominal 0.90): a two-parameter linear forward model cannot generate spread when model structural error dominates parameter uncertainty. The EnKF, operating in state space rather than parameter space, was the only method with positive skill score and KGE. Reported the universally negative NSE rather than omitting it.
- Trained a multivariate LSTM (TensorFlow, 22,726 parameters, 12-week lookback over six state variables) as a surrogate forward operator for a 50-member EnKF, because the source hydrological model emits a static trajectory and cannot be stepped inside a filter.
- Shipped as an installable package rather than notebooks: plugin architecture (abstract base class, result dataclass, method registry) so new methods drop in without touching existing code, a seeded-RNG reproducibility contract enforced in tests, QA/QC checks for gaps, stuck sensors, and rate spikes, a CLI, and a pytest/Hypothesis property-based suite at 87% coverage. Published as a public Jupyter Book: 15 executed notebooks and a 12-section engineering report.

Credit-Risk ML, Loan-Default Prediction

Python, scikit-learn

- Benchmarked 14 models on 32,416 loans with an explicit train-vs-test overfit check; selected Gradient Boosting at 0.95 test ROC-AUC with a small generalization gap.
- Added probability calibration (Brier 0.05), ROC/PR curves, partial dependence, and permutation importance; fully reproducible (seed 42, pinned versions).

CapSight AI, LLM Pitch-Deck Grading Platform

React, FastAPI, PostgreSQL, Node/TypeScript, OpenAI

- Built a three-service application that scores uploaded pitch decks across six categories with an LLM, persisted to PostgreSQL with a history view.
- Enforced structured output via OpenAI `json_schema` and made grading deterministic (seed 42, temperature 0) for repeatable results.
- Hardened the grading core with adaptive retry, score clamping, an unreadable-deck sentinel, a fallback result, and non-blocking database writes so grading still returns if the database is down.

Reanalysis Dashboard

FastAPI, Alpine.js, TensorFlow

- Generalized the capstone pipeline into a self-serve tool: upload a model CSV and an observation CSV, pick the target column, and run an LSTM + Ensemble Kalman Filter reanalysis locally, with a server-sent-events progress stream during training and downloadable outputs with confidence intervals.

Additional: UFC relational database (five-table normalized PostgreSQL schema); Selenium/TestNG end-to-end UI suite (eight areas, Java/Maven); Pro-Code AI Socratic programming tutor (FastAPI, PostgreSQL, React).